

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
ĐẠI HỌC QUỐC GIA HÀ NỘI

—o0o—



BÁO CÁO CHUẨN LẬP TRÌNH CÁC THUẬT TOÁN

Giảng viên: TS.GVC Lê Phê Đô

Lớp học phần: INT3102 2

Sinh viên: Nguyễn Thị Hải Yến

MSV: 23021756

Lớp: K68I-CS2

Hà Nội, 2025

Mục lục

1. Giải phương trình gần đúng.....	3
1.1. Phương pháp chia đôi.....	3
1.2. Phương pháp Newton.....	5
1.3. Phương pháp dây cung.....	8
1.4. Phương pháp điểm sai.....	10
2. Giải chính xác hệ phương trình.....	13
2.1. Phương pháp Crammer.....	13
2.2. Phương pháp khử Gauss.....	17
2.3. Phương pháp phân tích LU.....	21
3. Giải gần đúng hệ phương trình.....	25
3.1. Phương pháp lặp Gauss – Seidel.....	25
3.2. Phương pháp lặp Jacobi.....	30
4. Phương pháp nội suy.....	33
4.1. Nội suy Lagrange.....	33
4.2. Nội suy Newton.....	36
5. Phương pháp nội suy Spline.....	39
6. Phương pháp bình phương tối thiểu.....	43
7. Bài toán trị riêng.....	47

1. Giải phương trình gần đúng

1.1. Phương pháp chia đôi

a. Thuật toán:

```
import math

def f(x, coefficients):
    val = 0
    degree = len(coefficients) - 1
    for i, coef in enumerate(coefficients):
        val += coef * (x ** (degree - i))
    return val

def bisection_method(coefficients, a, b, tolerance):
    fa = f(a, coefficients)
    fb = f(b, coefficients)

    if fa * fb >= 0:
        return None, "Lỗi: Hàm không đổi dấu trên đoạn [{0}, {1}].".format(a, b)

    iteration = 0
    max_iter_estimate = math.log2((b - a) / tolerance)

    print(f"{'Lần lặp':<10} | {'a':<18} | {'b':<18} | {'Nghiem c':<18} | {'f(c)':<18} | {'Sai số (b-a)':<18}")
    print("-" * 110)

    while (b - a) / 2 > tolerance:
        iteration += 1
```

```

    c = (a + b) / 2
    fc = f(c, coefficients)

    print(f"{iteration:<10} | {a:<18.12f} | {b:<18.12f} | {c:<18.12f} | {fc:<18.12e} | {(b-a):<18.12e}")

    if fc == 0:
        return c, iteration

    if f(a, coefficients) * fc < 0:
        b = c
    else:
        a = c

    return (a + b) / 2, iteration

# --- CẤU HÌNH BÀI TOÁN THEO YÊU CẦU ---
# Phương trình bậc 10:  $x^{10} - x - 1 = 0$ 
# Hệ số: [1, 0, 0, 0, 0, 0, 0, 0, 0, -1, -1]
coeffs_poly_10 = [1, 0, 0, 0, 0, 0, 0, 0, 0, -1, -1]

# Khoảng cách ly nghiệm (cần tự xác định trước hoặc khảo sát hàm số)
a_start = 1.0
b_start = 2.0

# Độ chính xác yêu cầu  $10^{-10}$ 
epsilon = 1e-10

```

```

root, steps = bisection_method(coeffs_poly_10, a_start, b_start,
epsilon)

print("-" * 110)
if root:
    print(f"Kết quả: Nghiệm x ≈ {root:.12f}")
    print(f"Số bước lặp: {steps}")
else:
    print(steps)

```

b. Đánh giá sai số:

- Tại bước lặp thứ n , sai số tuyệt đối của nghiệm gần đúng c_n so với nghiệm thực x^* thỏa mãn:

$$|c_n - x^*| \leq \frac{b_0 - a_0}{2^n}$$

- Số lần lặp cần thiết để đạt độ chính xác $\varepsilon = 10^{-10}$:

$$\frac{b_0 - a_0}{2^n} < \varepsilon \Rightarrow n > \log_2\left(\frac{b_0 - a_0}{\varepsilon}\right)$$

1.2. Phương pháp Newton

a. Thuật toán

```

import math

def f(x, coefficients):
    """Tính giá trị f(x)"""
    val = 0
    degree = len(coefficients) - 1

```

```

    for i, coef in enumerate(coefficients):
        val += coef * (x ** (degree - i))
    return val

def get_derivative_coeffs(coefficients):
    deriv_coeffs = []
    degree = len(coefficients) - 1
    for i in range(degree):
        deriv_coeffs.append(coefficients[i] * (degree - i))
    return deriv_coeffs

def newton_method(coeffs, x0, tolerance):
    deriv_coeffs = get_derivative_coeffs(coeffs)
    x_current = x0
    iteration = 0

    print(f"\n--- PHƯƠNG PHÁP NEWTON (Bắt đầu tại x0={x0}) ---")
    print(f"{'Lần lặp':<10} | {'Nghiem x':<20} | {'Sai số ước lượng':<20}")
    print("-" * 60)

    while True:
        iteration += 1
        fx = f(x_current, coeffs)
        f_prime_x = f(x_current, deriv_coeffs)

        if f_prime_x == 0:
            print("Lỗi: Đạo hàm bằng 0, không thể chia.")

```

```

        return None

    # Công thức Newton
    x_next = x_current - (fx / f_prime_x)

    error = abs(x_next - x_current)
    print(f"{iteration:<10} | {x_next:<20.12f} | {error:<20.12e}")

    if error < tolerance:
        return x_next, iteration

    x_current = x_next

    if iteration > 100: # Tránh lặp vô hạn
        print("Không hội tụ sau 100 bước.")
        return None

# --- THỰC THI ---
# Phương trình: x^10 - x - 1 = 0
coeffs_poly_10 = [1, 0, 0, 0, 0, 0, 0, 0, 0, -1, -1]
epsilon = 1e-10

root_newton, step_newton = newton_method(coeffs_poly_10, 1.5, epsilon)
if root_newton:
    print(f">>> Kết quả Newton: x ≈ {root_newton:.12f} (Số bước: {step_newton})")

```

b. Đánh giá sai số:

$$|x_{n+1} - x_n| < \varepsilon \text{ hoặc } f(x_{n+1}) < \varepsilon$$

1.3. Phương pháp dây cung

a. Thuật toán:

```
def f(x, coefficients):
    val = 0
    degree = len(coefficients) - 1
    for i, coef in enumerate(coefficients):
        val += coef * (x ** (degree - i))
    return val

def secant_method_standard(coefficients, x0, x1, tolerance):
    print(f"\n--- BẮT ĐẦU PHƯƠNG PHÁP DÂY CUNG ---")
    print(f"Khoảng khởi tạo: x0 = {x0}, x1 = {x1}")
    print(f"{'Bước':<5} | {'x_k-1':<18} | {'x_k':<18} | {'x_k+1 (Mới)':<18} | {'Sai số':<18}")
    print("-" * 85)

    iteration = 0

    while True:
        iteration += 1
        fx0 = f(x0, coefficients)
        fx1 = f(x1, coefficients)

        if fx1 - fx0 == 0:
            print("Lỗi: f(x1) = f(x0), không thể chia.")
            return None, iteration

    # Áp dụng công thức Dây cung
```



```

x_new = x1 - fx1 * (x1 - x0) / (fx1 - fx0)

# Tính sai số gần đúng
error = abs(x_new - x1)

print(f"{iteration:<5} | {x0:<18.10f} | {x1:<18.10f} | {x_new:<18.10f} | {error:<18.2e}")

if error < tolerance:
    return x_new, iteration

x0 = x1
x1 = x_new

if iteration > 1000:
    print("Không hội tụ sau 1000 bước.")
    return None, iteration

# Phương trình:  $x^{10} - x - 1 = 0$ 
coeffs = [1, 0, 0, 0, 0, 0, 0, 0, 0, -1, -1]
epsilon = 1e-10

x_start_1 = 1.0
x_start_2 = 2.0

root, steps = secant_method_standard(coeffs, x_start_1, x_start_2, epsilon)

print("-" * 85)

```

```
if root:
    print(f"KẾT QUẢ: Nghiệm x ≈ {root:.12f}")
    print(f"Tổng số bước lặp: {steps}")
```

b. Đánh giá sai số:

$$|x_{n+1} - x_n| < \varepsilon$$

1.4. Phương pháp điểm sai

a. Thuật toán

```
import math

def f(x, coefficients):
    val = 0
    degree = len(coefficients) - 1
    for i, coef in enumerate(coefficients):
        val += coef * (x ** (degree - i))
    return val

def false_position_method(coefficients, a, b, tolerance):
    fa = f(a, coefficients)
    fb = f(b, coefficients)

    if fa * fb >= 0:
        print(f"Lỗi: Hàm không đổi dấu trên đoạn [{a}, {b}].")
        return None, 0
```

```

print(f"\n--- PHƯƠNG PHÁP ĐIỂM SAI (FALSE POSITION) ---")
print(f"Khoảng ban đầu: [{a}, {b}]")
print(f"{'Lần lặp':<10} | {'a':<18} | {'b':<18} | {'Nghiệm c':<18} | {'f(c)':<18}")
print("-" * 90)

iteration = 0
c_old = a # Lưu giá trị c cũ để tính sai số hội tụ

while True:
    iteration += 1

    # Tính giá trị hàm tại 2 đầu mút hiện tại
    fa = f(a, coefficients)
    fb = f(b, coefficients)

    c = (a * fb - b * fa) / (fb - fa)
    fc = f(c, coefficients)

    print(f"{iteration:<10} | {a:<18.10f} | {b:<18.10f} | {c:<18.10f} | {fc:<18.2e}")

    if abs(fc) < tolerance or abs(c - c_old) < tolerance:
        return c, iteration
    if fa * fc < 0:
        b = c
    else:
        a = c

```

```

        c_old = c

    if iteration > 1000:
        print("Không hội tụ sau 1000 bước (Cảnh báo: Điểm sai hội
tụ chậm tại một đầu mút).")
        return c, iteration

# Phương trình: x^10 - x - 1 = 0
coeffs_poly_10 = [1, 0, 0, 0, 0, 0, 0, 0, 0, -1, -1]
epsilon = 1e-10

a_start = 1.0
b_start = 2.0

root, steps = false_position_method(coeffs_poly_10, a_start, b_start,
epsilon)

print("-" * 90)
if root:
    print(f"KẾT QUẢ: Nghiệm x ≈ {root:.12f}")
    print(f"Số bước lặp: {steps}")

```

b. Đánh giá sai số:

$$|x_{n+1} - x_n| < \varepsilon$$

2. Giải chính xác hệ phương trình

2.1. Phương pháp Crammer

a. Thuật toán

```
import random
import copy

def print_matrix(A):
    n = len(A)
    print(f"Ma trận {n}x{n} (Hiển thị 5x5 đầu tiên):")
    for i in range(min(n, 5)):
        row_str = " ".join(f"{val:8.2f}" for val in A[i][:5])
        print(f"[{row_str} ... ]")
    print("..." * 10)

def gaussian_determinant(matrix):
    n = len(matrix)
    # Tạo bản sao để không làm thay đổi ma trận gốc
    temp_mat = copy.deepcopy(matrix)
    det = 1.0

    for i in range(n):
        # 1. Tìm phần tử trụ (pivot) lớn nhất trong cột i để tránh sai số
        pivot_idx = i
        for j in range(i + 1, n):
            if abs(temp_mat[j][i]) > abs(temp_mat[pivot_idx][i]):
                pivot_idx = j
```

```

        if i != pivot_idx:
            temp_mat[i], temp_mat[pivot_idx] = temp_mat[pivot_idx],
temp_mat[i]
            det *= -1 # Hoán đổi dòng làm đổi dấu định thức

        pivot = temp_mat[i][i]

        if pivot == 0:
            return 0.0 # Định thức bằng 0

        det *= pivot

        for j in range(i + 1, n):
            factor = temp_mat[j][i] / pivot
            for k in range(i, n): # Chỉ cần chạy từ i trở đi
                temp_mat[j][k] -= factor * temp_mat[i][k]

    return det

def crammer_solver(A, B):
    n = len(A)
    print(f"\n--- BẮT ĐẦU GIẢI CRAMMER (n={n}) ---")

    # Bước 1: Tính định thức D của ma trận hệ số A
    D = gaussian_determinant(A)
    print(f"Định thức D = {D:.4e}")

    if abs(D) < 1e-15: # So sánh với số rất nhỏ thay vì 0

```

```
        return None, "Định thức  $D \approx 0$ . Hệ phương trình vô nghiệm hoặc vô số nghiệm."
```

```
solutions = []
```

```
# Bước 2: Tính các định thức  $D_i$  và tìm nghiệm  $x_i$ 
```

```
# Duyệt qua từng cột để thay thế bằng vector B
```

```
for i in range(n):
```

```
    # Tạo ma trận  $A_i$  bằng cách thay cột  $i$  của  $A$  bằng  $B$ 
```

```
     $A_i = \text{copy.deepcopy}(A)$ 
```

```
    for row in range(n):
```

```
         $A_i[\text{row}][i] = B[\text{row}]$ 
```

```
    # Tính định thức  $D_i$ 
```

```
     $D_i = \text{gaussian\_determinant}(A_i)$ 
```

```
    # Tính nghiệm  $x_i$ 
```

```
     $x_i = D_i / D$ 
```

```
    solutions.append( $x_i$ )
```

```
    if (i + 1) % 5 == 0 or i == n - 1:
```

```
        print(f"Đã tính xong  $x_{i+1}$ ...")
```

```
return solutions, "Thành công"
```

```
n = 30
```

```
A_matrix = [[random.uniform(-10, 10) for _ in range(n)] for _ in range(n)]
```

```
B_vector = [random.uniform(-10, 10) for _ in range(n)]
```

```

print("Đã khởi tạo hệ phương trình ngẫu nhiên 30 ẩn.")
print_matrix(A_matrix)

# 2. Gọi hàm giải
nghiem, status = crammer_solver(A_matrix, B_vector)

# 3. Xuất kết quả
print("-" * 50)
if nghiem:
    print("KẾT QUẢ NGHIỆM:")
    for i in range(n):
        print(f"x_{i+1:<2} = {nghiem[i]:.10f}")

    # Kiểm tra lại nghiệm đầu tiên (thử lại Ax - B xem có ≈ 0 không)
    # Tổng a_0j * x_j
    check_val = sum(A_matrix[0][j] * nghiem[j] for j in range(n))
    print(f"\nKiểm tra lại pt 1: Vế trái = {check_val:.5f}, Vế phải (B[0]) = {B_vector[0]:.5f}")
else:
    print(f"Lỗi: {status}")

```

b. Đánh giá sai số:

– Độ phức tạp:

- Mã nguồn trên sử dụng khử Gauss để tính định thức ($O(n^3)$)
- Crammer lặp lại việc này $n + 1$ lần.

- Tổng độ phức tạp là $O(n^4)$. Với $n = 30$, máy tính hiện đại xử lý trong tích tắc. Tuy nhiên, nếu n tăng lên 50 hoặc 100, phương pháp này sẽ chậm hơn rất nhiều so với phương pháp khử Gauss trực tiếp.
- Sai số:
- Phương pháp Crammer dễ bị sai số làm tròn lớn khi định thức D quá nhỏ hoặc quá lớn. Nếu bạn thấy kết quả "Kiểm tra lại" bị lệch nhiều, đó là do hạn chế của kiểu dữ liệu số thực (float) khi tính định thức của ma trận lớn.

2.2. Phương pháp khử Gauss

a. Thuật toán

```
import random
import copy

def generate_system(n):
    A = [[random.uniform(-100, 100) for _ in range(n)] for _ in range(n)]
    # Vế phải B
    B = [random.uniform(-100, 100) for _ in range(n)]
    return A, B

def gauss_elimination(A_in, B_in):
    n = len(B_in)
    A = copy.deepcopy(A_in)
    B = copy.deepcopy(B_in)
```

```

# 1. QUÁ TRÌNH THUẬN
for i in range(n):
    max_row = i
    for k in range(i + 1, n):
        if abs(A[k][i]) > abs(A[max_row][i]):
            max_row = k

    A[i], A[max_row] = A[max_row], A[i]
    B[i], B[max_row] = B[max_row], B[i]

    if abs(A[i][i]) < 1e-15:
        return None, "Ma trận suy biến (hoặc gần suy biến), không
thể chia."

# --- Khử Gauss ---
for k in range(i + 1, n):
    factor = A[k][i] / A[i][i]
    for j in range(i, n): # Tối ưu: chỉ chạy từ cột i
        A[k][j] -= factor * A[i][j]
    B[k] -= factor * B[i]

# 2. QUÁ TRÌNH NGƯỢC
x = [0.0] * n
for i in range(n - 1, -1, -1):
    sum_ax = sum(A[i][j] * x[j] for j in range(i + 1, n))
    x[i] = (B[i] - sum_ax) / A[i][i]

```

```

    return x, "Thành công"

def evaluate_error(A, B, x):
    n = len(B)
    max_error = 0.0

    print("\n--- ĐÁNH GIÁ SAI SỐ (Residual Check) ---")
    print(f"{'PT số':<10} | {'Vế phải thực (B)':<20} | {'Vế trái tính toán (Ax)':<20} | {'Sai lệch (Residual)':<20}")
    print("-" * 80)

    indices_to_print = list(range(5)) + list(range(n-5, n))

    for i in range(n):
        # Tính giá trị vế trái: sum(A_ij * x_j)
        Ax_i = sum(A[i][j] * x[j] for j in range(n))
        residual = abs(B[i] - Ax_i)

        if residual > max_error:
            max_error = residual

        if i in indices_to_print:
            print(f"{i+1:<10} | {B[i]:<20.5f} | {Ax_i:<20.5f} | {residual:<20.2e}")
            elif i == 5:
                print(f"{'...':<10} | {'...':<20} | {'...':<20} | {'...':<20}")

    return max_error

```

```

N = 50

print(f"Đang khởi tạo hệ phương trình {N}x{N}...")
matrix_A, vector_B = generate_system(N)

solution, status = gauss_elimination(matrix_A, vector_B)

if solution:
    print(f"Giải thành công! (Hiển thị 5 nghiệm đầu):
{solution[:5]} ...")

    max_err = evaluate_error(matrix_A, vector_B, solution)

    print("-" * 80)
    print(f"KẾT LUẬN ĐÁNH GIÁ SAI SỐ:")
    print(f"Sai số dư lớn nhất (Max Residual): {max_err:.5e}")

    if max_err < 1e-9:
        print("=> Nghiệm RẤT CHÍNH XÁC (sai số không đáng kể).")
    elif max_err < 1e-5:
        print("=> Nghiệm CHẤP NHẬN ĐƯỢC.")
    else:
        print("=> Cảnh báo: Sai số khá lớn, ma trận có thể điều kiện
xấu (ill-conditioned).")
else:
    print(f"Lỗi: {status}")

```

b. Đánh giá sai số

- Đối với phương pháp giải đúng (Direct Method) như Gauss, sai số chủ yếu đến từ việc làm tròn số của máy tính (floating-point arithmetic). Ta đánh giá sai số dựa trên Vector dư (Residual Vector).
- Gọi x^* là nghiệm tính toán được, vector dư r được tính bằng:

$$r = B - A \cdot x^*$$

- Sai số của phép giải được đánh giá qua chuẩn của vector dư (thường dùng chuẩn vô cùng - giá trị lớn nhất):

$$\|r\|_{\infty} = \max_{1 \leq i \leq n} |r_i| = \max_{1 \leq i \leq n} |b_i - \sum_{j=1}^n a_{ij} x_j^*|$$

- Nếu $\|r\|_{\infty} \approx 0$ (ví dụ $< 10^{-10}$), nghiệm được coi là chính xác.

2.3. Phương pháp phân tích LU

a. Thuật toán

```
import random

def generate_system(n):
    A = [[0.0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            A[i][j] = random.uniform(-10, 10)
        A[i][i] = sum(abs(A[i][j]) for j in range(n)) +
            random.uniform(1, 10)
```

```

B = [random.uniform(-10, 10) for _ in range(n)]
return A, B

def lu_decomposition(A):
    n = len(A)

    L = [[0.0] * n for _ in range(n)]
    U = [[0.0] * n for _ in range(n)]

    for i in range(n):
        # 1. Tính dòng i của U
        for k in range(i, n):
            sum_u = sum(L[i][j] * U[j][k] for j in range(i))
            U[i][k] = A[i][k] - sum_u

        for k in range(i, n):
            if i == k:
                L[i][i] = 1.0 # Đường chéo của L luôn là 1
            else:
                sum_l = sum(L[k][j] * U[j][i] for j in range(i))
                # Kiểm tra chia cho 0
                if U[i][i] == 0:
                    return None, None, "Lỗi: Phần tử trụ bằng 0 (cần
đổi dòng - Pivot)."
                L[k][i] = (A[k][i] - sum_l) / U[i][i]

    return L, U, "Thành công"

```

```

def solve_lu(L, U, B):
    n = len(B)

    # Bước 1: Giải  $Ly = B$  (Thế thuận)
    y = [0.0] * n
    for i in range(n):
        sum_ly = sum(L[i][j] * y[j] for j in range(i))
        y[i] = B[i] - sum_ly # Vì  $L[i][i]$  luôn = 1 nên không cần chia

    # Bước 2: Giải  $Ux = y$  (Thế ngược)
    x = [0.0] * n
    for i in range(n - 1, -1, -1):
        sum_ux = sum(U[i][j] * x[j] for j in range(i + 1, n))
        if U[i][i] == 0:
            return None
        x[i] = (y[i] - sum_ux) / U[i][i]

    return x

def evaluate_error(A, B, x):
    n = len(B)
    max_residual = 0.0

    print(f"\n{'PT số':<10} | {'Vế trái (Ax)':<20} | {'Vế phải (B)':<20} | {'Sai số (Residual)':<20}")
    print("-" * 80)

```

```

indices = list(range(5)) + list(range(n-5, n))

for i in range(n):
    Ax_i = sum(A[i][j] * x[j] for j in range(n))
    res = abs(B[i] - Ax_i)
    if res > max_residual:
        max_residual = res

    if i in indices:
        print(f"{i+1:<10} | {Ax_i:<20.5f} | {B[i]:<20.5f} | {res:<20.2e}")
    elif i == 5:
        print(f"'...':<10} | {'...':<20} | {'...':<20} | {'...':<20}")

return max_residual

N = 50
print(f"--- PHƯƠNG PHÁP LU (DOOLITTLE) CHO HỆ {N} ẨN ---")

# 1. Tạo hệ phương trình
matrix_A, vector_B = generate_system(N)

# 2. Phân tích LU
L_mat, U_mat, status = lu_decomposition(matrix_A)

if L_mat and U_mat:
    print("Phân tích LU thành công.")

```



```

# 3. Giải hệ phương trình
solution = solve_lu(L_mat, U_mat, vector_B)

if solution:
    print(f"Tìm được nghiệm (5 giá trị đầu): {[round(v, 5) for v
in solution[:5]]} ...")

    # 4. Đánh giá sai số
    max_err = evaluate_error(matrix_A, vector_B, solution)
    print("-" * 80)
    print(f"TỔNG KẾT SAI SỐ (Max Residual): {max_err:.5e}")
else:
    print("Lỗi trong quá trình giải ngược/xuôi.")
else:
    print(f"Lỗi phân tích: {status}")

```

b. Đánh giá sai số

- Trong kết quả chạy thực tế, cột "Sai số (Residual)" thường rất nhỏ (khoảng 10^{-13} đến 10^{-15}).
- Nếu sai số nhỏ hơn 10^{-6} , kết quả hoàn toàn chấp nhận được cho các bài toán kỹ thuật.

3. Giải gần đúng hệ phương trình

3.1. Phương pháp lặp Gauss – Seidel

a. Thuật toán

```
import random
```

```

import math
import time

def generate_large_system(n):
    print(f"Đang khởi tạo dữ liệu cho hệ {n}x{n} (có thể mất vài giây)...")
    A = []
    B = []

    for i in range(n):
        row = []
        row_sum = 0
        for j in range(n):
            if i == j:
                val = 0 # Sẽ gán sau
            else:
                val = random.uniform(-1, 1)
                row_sum += abs(val)
            row.append(val)

        row[i] = row_sum + random.uniform(10, 20)
        A.append(row)

        B.append(random.uniform(-100, 100))

    return A, B

def gauss_seidel(A, B, tolerance=1e-10, max_iterations=1000):

```

```

n = len(B)
x = [0.0] * n # Khởi tạo nghiệm ban đầu toàn 0

print(f"\n--- BẮT ĐẦU LẶP GAUSS-SEIDEL (n={n}) ---")
print(f"{'Lần lặp':<10} | {'Sai số lặp (Norm)':<20} | {'Trạng thái'}")
print("-" * 60)

start_time = time.time()

for k in range(max_iterations):
    max_diff = 0.0 # Sai số lớn nhất giữa 2 lần lặp liên tiếp

    for i in range(n):
        sigma = 0.0
        for j in range(n):
            if i != j:
                sigma += A[i][j] * x[j]

        x_new = (B[i] - sigma) / A[i][i]

        # Tính sai số hội tụ
        diff = abs(x_new - x[i])
        if diff > max_diff:
            max_diff = diff

    x[i] = x_new # Cập nhật ngay lập tức

```

```

        if (k + 1) % 10 == 0 or max_diff < tolerance:
            print(f"{k+1:<10} | {max_diff:<20.5e} | Đang chạy...")

        if max_diff < tolerance:
            end_time = time.time()
            print("-" * 60)
            print(f"Hội tụ thành công sau {k+1} bước lặp.")
            print(f"Thời gian tính toán: {end_time - start_time:.4f}
giây")

            return x, k+1

    print("Cảnh báo: Đã vượt quá số lần lặp tối đa mà chưa đạt độ
chính xác yêu cầu.")

    return x, max_iterations

def evaluate_residual(A, B, x):
    n = len(B)
    max_res = 0.0

    for i in range(n):
        Ax_i = 0.0
        for j in range(n):
            Ax_i += A[i][j] * x[j]

        res = abs(B[i] - Ax_i)
        if res > max_res:
            max_res = res

    return max_res

```

```

N = 2000
epsilon = 1e-10

A_matrix, B_vector = generate_large_system(N)

solution, steps = gauss_seidel(A_matrix, B_vector, epsilon)

if solution:
    print("\n--- ĐÁNH GIÁ KẾT QUẢ ---")
    print(f" NGHIỆM TẠI 5 ẨN ĐẦU TIÊN: {[round(v, 6) for v in
solution[:5]]} ...")

    # Tính sai số dư thực tế
    residual_error = evaluate_residual(A_matrix, B_vector, solution)
    print(f"Sai số dư lớn nhất (Max Residual ||B - Ax||):
{residual_error:.5e}")

    if residual_error < 1e-6:
        print("=> Kết quả RẤT TỐT.")
    else:
        print("=> Kết quả CHẤP NHẬN ĐƯỢC (có thể cần thêm vòng lặp).")

```

b. Đánh giá sai số

- Sai số lặp: $\|x^{(k+1)} - x^{(k)}\| < \epsilon$. Dùng để dừng vòng lặp.
- Sai số dư: $\|B - Ax^*\|$. Dùng để đánh giá độ chính xác của nghiệm tìm được.

3.2. Phương pháp lặp Jacobi

a. Thuật toán

```
import random
import time
import copy

def generate_large_system(n):
    print(f"Đang khởi tạo dữ liệu cho hệ {n}x{n} (có thể mất vài giây)...")

    A = []
    B = []

    for i in range(n):
        row = []
        row_sum = 0
        for j in range(n):
            if i == j:
                val = 0
            else:
                val = random.uniform(-1, 1)
                row_sum += abs(val)
            row.append(val)

        row[i] = row_sum + random.uniform(10, 20)
        A.append(row)
        B.append(random.uniform(-100, 100))
```

```

return A, B

def jacobi_method(A, B, tolerance=1e-10, max_iterations=1000):
    n = len(B)
    x_old = [0.0] * n # Giá trị tại bước k
    x_new = [0.0] * n # Giá trị tại bước k+1

    print(f"\n--- BẮT ĐẦU LẶP JACOBI (n={n}) ---")
    print(f"{'Lần lặp':<10} | {'Sai số lặp (Norm)':<20} | {'Trạng thái'}")
    print("-" * 60)

    start_time = time.time()

    for k in range(max_iterations):
        max_diff = 0.0 # Sai số lớn nhất trong lần lặp này

        # Duyệt qua từng phương trình
        for i in range(n):
            sigma = 0.0
            # Tính tổng a_ij * x_j(old)
            for j in range(n):
                if i != j:
                    sigma += A[i][j] * x_old[j]

            # Tính x_new[i]
            val = (B[i] - sigma) / A[i][i]
            x_new[i] = val

```

```

        diff = abs(val - x_old[i])
        if diff > max_diff:
            max_diff = diff

    if (k + 1) % 10 == 0 or max_diff < tolerance:
        print(f"{k+1:<10} | {max_diff:<20.5e} | Đang chạy...")

    # Kiểm tra điều kiện dừng
    if max_diff < tolerance:
        end_time = time.time()
        print("-" * 60)
        print(f"Hội tụ thành công sau {k+1} bước lặp.")
        print(f"Thời gian tính toán: {end_time - start_time:.4f}
giây")
        return x_new, k+1

    x_old = x_new[:]

    print("Cảnh báo: Không hội tụ sau số bước tối đa.")
    return x_new, max_iterations

N = 2000
epsilon = 1e-10

# 1. Tạo dữ liệu
A_matrix, B_vector = generate_large_system(N)

```



```
# 2. Giải bằng Jacobi
solution, steps = jacobi_method(A_matrix, B_vector, epsilon)

# 3. Kết quả sơ bộ
if solution:
    print(f"\nKết quả 5 nghiệm đầu: {[round(v, 5) for v in
solution[:5]]} ...")
```

b. Đánh giá sai số

- Sai số dư: $\|B - Ax^*\|$. Dùng để đánh giá độ chính xác của nghiệm tìm được.

4. Phương pháp nội suy

4.1. Nội suy Lagrange

a. Thuật toán

```
import math

def generate_data(n):
    """
    Tạo n điểm mốc (x, y) dựa trên hàm y = sin(x).
    Khoảng [0, 10].
    """
    x_points = []
    y_points = []
    step = 10.0 / (n - 1)
    for i in range(n):
```

```

        x = i * step
        x_points.append(x)
        y_points.append(math.sin(x))
    return x_points, y_points

def lagrange_interpolation(x_query, x_points, y_points):
    n = len(x_points)
    result = 0.0

    # Áp dụng công thức Lagrange  $P(x) = \sum(y_i * L_i(x))$ 
    for i in range(n):
        term = y_points[i]

        # Tính  $L_i(x)$ 
        for j in range(n):
            if i != j:
                if x_points[i] - x_points[j] == 0:
                    continue
                term = term * (x_query - x_points[j]) / (x_points[i] -
x_points[j])

        result += term

    return result

[cite_start]N = 50

# 1. Khởi tạo dữ liệu mẫu (50 điểm trên đồ thị hình sin)

```

```
X_nodes, Y_nodes = generate_data(N)

print(f"--- NỘI SUY LAGRANGE VỚI {N} ĐIỂM MỐC ---")
print(f"Dữ liệu mốc: x từ {X_nodes[0]:.2f} đến {X_nodes[-1]:.2f}")

# 2. Chọn điểm cần nội suy (nằm giữa các mốc để kiểm tra)
# Chọn x = 2.55 (nằm giữa các mốc nhưng không trùng mốc nào)
val_check = 2.55
true_val = math.sin(val_check)

# 3. Tính toán
interpolated_val = lagrange_interpolation(val_check, X_nodes, Y_nodes)

# 4. Xuất kết quả và Đánh giá sai số
print("-" * 60)
print(f"Tại x = {val_check}:")
print(f"Giá trị thực (sin(x)):      {true_val:.10f}")
print(f"Giá trị nội suy Lagrange:    {interpolated_val:.10f}")

error = abs(true_val - interpolated_val)
print(f"Sai số tuyệt đối:          {error:.5e}")

# Cảnh báo về hiện tượng Runge với 50 điểm
print("-" * 60)
print("NHẬN XÉT:")
if error < 1e-6:
    print("Nội suy chính xác tại điểm kiểm tra nằm giữa vùng trung tâm.")
```

```

else:
    print("Sai số lớn (do bậc đa thức quá cao).")

# Thử nghiệm tại điểm gần biên (để thấy hiện tượng dao động mạnh)
x_edge = 0.1
y_edge_interp = lagrange_interpolation(x_edge, X_nodes, Y_nodes)
print(f"\nKiểm tra tại vùng biên (x={x_edge}):")
print(f"Thực: {math.sin(x_edge):.5f} | Nội suy: {y_edge_interp:.5f}")

```

b. Đánh giá sai số

- Độ chính xác ở trung tâm: Tại các điểm nằm giữa khoảng (ví dụ $x = 5$ trong đoạn $[0, 10]$), kết quả thường khá chính xác.
- Hiện tượng Runge: Ở gần hai đầu mút của đoạn nội suy (ví dụ gần 0 hoặc gần 10), sai số sẽ tăng vọt cực kỳ lớn.
 - Đa thức bậc cao (bậc 49) có xu hướng dao động dữ dội ở hai đầu mút.
 - Lagrange không phù hợp cho số lượng điểm mốc lớn.

4.2. Nội suy Newton

a. Thuật toán

```

import math

import numpy as np # Sử dụng numpy để xử lý mảng tiện lợi hơn

def generate_data(n):

```

```

"""
Tạo n điểm mốc (x, y) dựa trên hàm  $y = \sin(x)$ .
Khoảng  $[0, 10]$ .
"""

x_points = np.linspace(0, 10, n)
y_points = np.sin(x_points)
return x_points, y_points

def divided_differences(x_points, y_points):
    n = len(y_points)
    coef = np.zeros([n, n])
    coef[:, 0] = y_points

    # Tính toán các cột tiếp theo
    for j in range(1, n):
        for i in range(n - j):
            # Công thức tỷ hiệu: (Sau - Trước) / (x_cuối - x_đầu)
            numerator = coef[i+1][j-1] - coef[i][j-1]
            denominator = x_points[i+j] - x_points[i]
            coef[i][j] = numerator / denominator

    return coef[0, :]

def newton_evaluate(coef, x_points, x_query):
    n = len(coef) - 1
    p = coef[n]

    for k in range(1, n + 1):

```

```

        p = coef[n - k] + (x_query - x_points[n - k]) * p

    return p

N = 50
# 1. Tạo dữ liệu
X_nodes, Y_nodes = generate_data(N)

print(f"--- NỘI SUY NEWTON VỚI {N} ĐIỂM MỐC ---")

# 2. Tính các hệ số (Tỷ hiệu)
coefficients = divided_differences(X_nodes, Y_nodes)

# 3. Kiểm tra tại một điểm
val_check = 2.55 # Điểm nằm giữa vùng trung tâm
true_val = math.sin(val_check)
interp_val = newton_evaluate(coefficients, X_nodes, val_check)

print(f"Tại x = {val_check}:")
print(f"Giá trị thực:      {true_val:.10f}")
print(f"Giá trị Newton:    {interp_val:.10f}")
print(f"Sai số tuyệt đối:  {abs(true_val - interp_val):.5e}")

# 4. Kiểm tra tại điểm biên (Hiện tượng Runge)
val_edge = 0.2
interp_edge = newton_evaluate(coefficients, X_nodes, val_edge)
print("-" * 60)
print(f"Tại vùng biên x = {val_edge}:")

```

```
print(f"Giá trị thực:      {math.sin(val_edge):.5f}")
print(f"Giá trị Newton:   {interp_edge:.5f}")
```

b. Đánh giá sai số

- Hiện tượng Runge: Ở gần hai đầu mút của đoạn nội suy (ví dụ gần 0 hoặc gần 10), sai số sẽ tăng vọt cực kỳ lớn.
 - Đa thức bậc cao (bậc 49) có xu hướng dao động dữ dội ở hai đầu mút.

5. Phương pháp nội suy Spline

a. Thuật toán

```
import math
import numpy as np

def generate_large_dataset(n):
    """
    Tạo 1000 điểm mốc dữ liệu từ hàm phức tạp hơn:  $y = \sin(x) + \cos(3x)$ .
    Khoảng  $[0, 20]$ .
    """
    x = np.linspace(0, 20, n)
    y = np.sin(x) + np.cos(3 * x)
    return x, y

def cubic_spline_coefficients(x, y):
    n = len(x) - 1
```

```

h = np.diff(x) # Khoảng cách giữa các điểm mốc  $h_i = x_{i+1} - x_i$ 

#  $a_i$  chính là giá trị  $y_i$ 
a = y[:-1]

alpha = np.zeros(n + 1)
l = np.zeros(n + 1)
mu = np.zeros(n + 1)
z = np.zeros(n + 1)

# Bước 1: Tính vế phải (rhs) alpha
for i in range(1, n):
    alpha[i] = (3 / h[i]) * (y[i+1] - y[i]) - (3 / h[i-1]) * (y[i]
- y[i-1])

# Bước 2: Giải hệ phương trình
# Điều kiện biên tự nhiên:  $l[0] = 1$ ,  $\mu[0] = 0$ ,  $z[0] = 0$ 
l[0] = 1.0
mu[0] = 0.0
z[0] = 0.0

for i in range(1, n):
    l[i] = 2 * (x[i+1] - x[i-1]) - h[i-1] * mu[i-1]
    mu[i] = h[i] / l[i]
    z[i] = (alpha[i] - h[i-1] * z[i-1]) / l[i]

# Điều kiện biên cuối:  $l[n] = 1$ ,  $z[n] = 0$ 
l[n] = 1.0

```



```

z[n] = 0.0

# Bước 3: Giải ngược để tìm c
c = np.zeros(n + 1)
b = np.zeros(n)
d = np.zeros(n)

c[n] = 0.0 # c_n = 0 (Natural spline)
for j in range(n - 1, -1, -1):
    c[j] = z[j] - mu[j] * c[j+1]

    # Tính b_j và d_j từ c_j và c_{j+1}
    b[j] = (y[j+1] - y[j]) / h[j] - h[j] * (c[j+1] + 2 * c[j]) / 3
    d[j] = (c[j+1] - c[j]) / (3 * h[j])

return a, b, c[:-1], d # c bỏ phần tử cuối vì chỉ cần n đoạn

def evaluate_spline(x_query, x_nodes, coeffs):
    a, b, c, d = coeffs
    n = len(x_nodes) - 1

    for i in range(n):
        if x_nodes[i] <= x_query <= x_nodes[i+1]:
            dx = x_query - x_nodes[i]
            val = a[i] + b[i]*dx + c[i]*(dx**2) + d[i]*(dx**3)
            return val

    return None # Ngoài khoảng nội suy

```

```
N_points = 1000 #
print(f"--- NỘI SUY CUBIC SPLINE VỚI {N_points} ĐIỂM MỐC ---")

# 1. Tạo dữ liệu lớn
X, Y = generate_large_dataset(N_points)
print(f"Đã tạo dữ liệu trên đoạn [{X[0]}, {X[-1]}] với hàm  $y = \sin(x) + \cos(3x)$ ")

# 2. Tính toán hệ số Spline (Training)
# Quá trình này rất nhanh nhờ giải thuật  $O(n)$ 
coeff_results = cubic_spline_coefficients(X, Y)
print("Đã tính xong bộ hệ số Spline.")

# 3. Kiểm tra và Đánh giá sai số
# Chọn một điểm kiểm tra nằm giữa 2 điểm mốc
# Ví dụ: Giữa điểm thứ 500 và 501
idx_test = 500
x_test = (X[idx_test] + X[idx_test+1]) / 2
true_val = math.sin(x_test) + math.cos(3 * x_test)

interp_val = evaluate_spline(x_test, X, coeff_results)

print("-" * 60)
print(f"Tại điểm kiểm tra  $x = {x_test:.5f}$ :")
print(f"Giá trị thực: {true_val:.10f}")
print(f"Giá trị Spline: {interp_val:.10f}")

error = abs(true_val - interp_val)
```

```

print(f"Sai số tuyệt đối: {error:.5e}")

# Nhận xét kết quả
if error < 1e-6:
    print("\n=> KẾT LUẬN: Phương pháp Spline cực kỳ chính xác và ổn
        định với 1000 điểm.")
else:
    print("\n=> KẾT LUẬN: Có lỗi xảy ra.")

```

b. Đánh giá sai số

- Sai số của Spline bậc 3 với 1000 điểm mốc thường rất nhỏ (cỡ 10^{-8} đến 10^{-10} hoặc nhỏ hơn tùy bước nhảy h).

6. Phương pháp bình phương tối thiểu

a. Thuật toán

```

import numpy as np
import random
import math

def generate_noisy_data(n):
    """
    Tạo 30 điểm dữ liệu giả lập có nhiễu.
    Mô hình gốc:  $y = 2x + 5 + \text{nhiều}$ 
    """
    x = np.linspace(0, 10, n)
    y = []

```

```

for val in x:
    noise = random.uniform(-2, 2)
    y.append(2 * val + 5 + noise)
return x, np.array(y)

def least_squares_fit(x, y, degree):
    n = len(x)
    k = degree + 1 # Số lượng hệ số cần tìm (ví dụ bậc 1 cần tìm a0,
a1)
    A = np.zeros((k, k))
    B = np.zeros(k)

    for row in range(k):
        for col in range(k):
            # Tính tổng các lũy thừa của x
            power = row + col
            sum_x = np.sum(x ** power)
            A[row][col] = sum_x

        # Tính vế phải
        sum_xy = np.sum(y * (x ** row))
        B[row] = sum_xy
    try:
        coeffs = np.linalg.solve(A, B)
        # coeffs trả về theo thứ tự [a0, a1, a2...] tương ứng a0 +
a1*x + ...
        return coeffs
    except np.linalg.LinAlgError:
        return None

```

```

def evaluate_fit(x, y, coeffs):
    n = len(x)
    y_pred = np.zeros(n)

    for i, a in enumerate(coeffs):
        y_pred += a * (x ** i)

    # 1. Sai số dư (Residuals)
    residuals = y - y_pred

    # 2. Tổng bình phương sai số (SSE)
    sse = np.sum(residuals ** 2)

    # 3. Sai số trung phương (RMSE) - Chỉ số quan trọng nhất
    rmse = math.sqrt(sse / n)

    return rmse, y_pred

N_points = 30
fit_degree = 1

print(f"--- BÌNH PHƯƠNG TỐI THIỂU (LEAST SQUARES) VỚI {N_points} ĐIỂM ---")

# 1. Tạo dữ liệu mẫu (30 điểm)
X_data, Y_data = generate_noisy_data(N_points)
print(f"Dữ liệu mẫu (5 điểm đầu): {Y_data[:5]}")

```

2. Thực hiện xấp xỉ

```
coefficients = least_squares_fit(X_data, Y_data, fit_degree)
```

```
if coefficients is not None:
```

```
    print("-" * 60)
```

```
    print(f"Kết quả tìm được (Đa thức bậc {fit_degree}):")
```

```
    # In phương trình tìm được
```

```
    equation_str = f"y ≈ {coefficients[0]:.4f}"
```

```
    for i in range(1, len(coefficients)):
```

```
        equation_str += f" + {coefficients[i]:.4f} * x^{i}"
```

```
    print(f"Phương trình hồi quy: {equation_str}")
```

3. Đánh giá sai số

```
rmse_val, y_predict = evaluate_fit(X_data, Y_data, coefficients)
```

```
print("-" * 60)
```

```
print("ĐÁNH GIÁ SAI SỐ:")
```

```
print(f"Sai số trung phương (RMSE): {rmse_val:.5f}")
```

```
# In bảng so sánh một vài điểm
```

```
print(f"\n{'x':<10} | {'y Thực tế':<15} | {'y Xấp xỉ':<15} | {'Sai  
lệch':<15}")
```

```
print("-" * 60)
```

```
indices = [0, 5, 10, 15, 20, 29] # Chọn một số điểm để in
```

```
for i in indices:
```

```
    diff = abs(Y_data[i] - y_predict[i])
```

```

        print(f"{X_data[i]:<10.2f} | {Y_data[i]:<15.4f} |  

        {y_predict[i]:<15.4f} | {diff:<15.4f}")

else:
    print("Lỗi: Không giải được hệ phương trình chuẩn (ma trận suy  

    biến).")

```

b. Đánh giá sai số

- Mục tiêu: Không phải là làm cho sai số bằng 0 (vì dữ liệu có nhiễu), mà là làm cho Tổng bình phương sai số (SSE) là nhỏ nhất có thể.
- RMSE (Root Mean Square Error): Trong kết quả chạy code, giá trị này cho biết độ lệch trung bình của các điểm dữ liệu so với đường cong tìm được.

7. Bài toán trị riêng

a. Thuật toán

```

import numpy as np
import random

def generate_symmetric_matrix(n):
    A = np.random.rand(n, n)
    A = (A + A.T) / 2
    return A

def power_iteration(A, tolerance=1e-10, max_iterations=1000):
    n = A.shape[0]

```

```

# 1. Khởi tạo vector ngẫu nhiên b_k (chuẩn hóa)
b_k = np.random.rand(n)
b_k = b_k / np.linalg.norm(b_k)

eigenvalue = 0

print(f"--- BẮT ĐẦU TÌM TRỊ RIÊNG (Power Iteration) ---")

for k in range(max_iterations):
    x_next = np.dot(A, b_k)

    b_k_next = x_next / np.linalg.norm(x_next)

    new_eigenvalue = np.dot(b_k_next.T, np.dot(A, b_k_next))

    if np.abs(new_eigenvalue - eigenvalue) < tolerance:
        return new_eigenvalue, b_k_next, k + 1

    eigenvalue = new_eigenvalue
    b_k = b_k_next

return eigenvalue, b_k, max_iterations

```

N = 100

```

# 1. Tạo ma trận
print(f"Đang khởi tạo ma trận {N}x{N}...")

```



```

matrix_A = generate_symmetric_matrix(N)

# 2. Thực hiện thuật toán Power Iteration
eig_val, eig_vec, steps = power_iteration(matrix_A)

print("-" * 60)
print(f"KẾT QUẢ SAU {steps} BƯỚC LẶP:")
print(f"Trị riêng trội tìm được: {eig_val:.10f}")

# 3. Đánh giá sai số (Kiểm tra  $Ax - \lambda x$ )
# Tính  $A * v$ 
Av = np.dot(matrix_A, eig_vec)
# Tính  $\lambda * v$ 
Lv = eig_val * eig_vec
# Tính sai số dư (Residual)
residual = Av - Lv
residual_norm = np.linalg.norm(residual)

print(f"Sai số dư  $\|Av - \lambda v\|$ : {residual_norm:.5e}")

if residual_norm < 1e-8:
    print("=> Kết quả chính xác.")
else:
    print("=> Kết quả chưa hội tụ tốt.")

# 4. So sánh với thư viện chuẩn (Numpy)
print("-" * 60)
print(f"KIỂM CHỨNG VỚI NUMPY (numpy.linalg.eigvalsh):")

```

```
# Hàm này tìm tất cả trị riêng
all_eigs = np.linalg.eigvalsh(matrix_A)
# Lấy trị riêng lớn nhất (về trị tuyệt đối)
max_eig_numpy = max(abs(np.min(all_eigs)), abs(np.max(all_eigs)))
print(f"Trị riêng lớn nhất từ Numpy: {max_eig_numpy:.10f}")
print(f"Chênh lệch với thuật toán tự viết: {abs(eig_val -
max_eig_numpy):.5e}")
```

b. Đánh giá sai số

- Sai số hội tụ dựa trên độ lệch $||Av - \lambda v||$. Với $N = 100$, thuật toán hội tụ nhanh và chính xác với các ma trận đối xứng.